

День 8 • 🎭 Сценарии для AI-агентов, знакомство с LangGraph

📋 Обсуждаемые темы:

- 🤖 Что такое продвинутый AI-агент?
- 🛠️ Инструменты: Travily и LangGraph
- 🧠 Ключевые концепции LangGraph (StateGraph, узлы, рёбра, память)
- 📋 Пример: агент-интегратор (Sheets → GitHub → LLM → Telegram)
- 💡 Идеи для расширения

Что такое продвинутый AI-агент?

AI-агент второго уровня — это не просто ответчик на запрос, а **активный исполнитель сценария**, который:

-  обрабатывает запрос,
-  обращается к нескольким API,
-  принимает решения на основе промежуточных данных,
-  действует по заданной логике,
-  формирует итоговую сводку / документ / команду

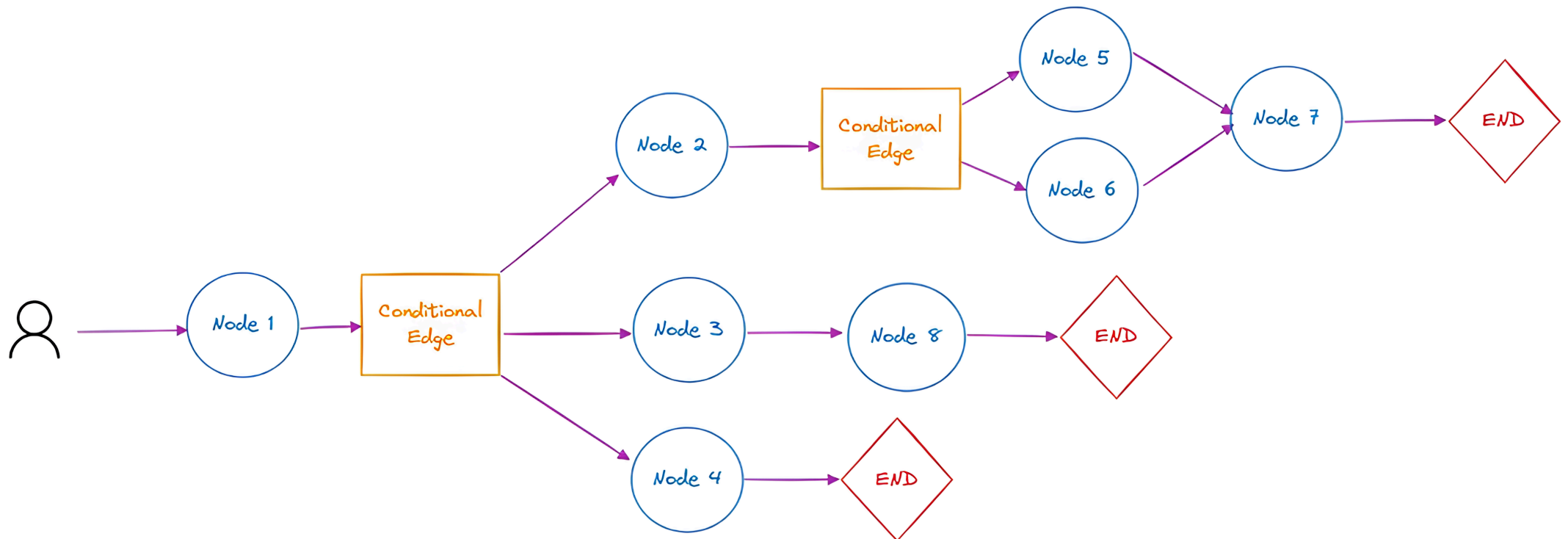
-  Низкоуровневый workflow-фреймворк на Python
-  Позволяет строить цепочки действий с условиями, ветвлениями, хранилищем состояния
-  Подходит для асинхронных задач и периодических сценариев



Инструменты для построения сценариев: LangGraph

-  Настройка над `LangChain`
-  Граф состояний (StateGraph): поток данных по узлам и рёбрам, а не линейная цепочка
-  Ветвления, циклы, условные переходы — полный контроль над сценарием
-  Внешние инструменты, несколько LLM, память между вызовами

🔧 Пример схемы графов LangGraph





LangGraph: четыре ключевые концепции

Концепция:	Назначение:
StateGraph	Граф, по которому движется поток данных
State	Общая структура данных между узлами (сообщения, контекст)
Node	Функция: принимает состояние → возвращает обновление
Edge	Связь между узлами: безусловная или условная (маршрутизация)



LangGraph: от графа к запуску

- **Узлы и рёбра:** `add_node` , `add_edge` , `add_conditional_edges` , `set_entry_point`
- **Компиляция:** `graph.compile()` — описание превращается в исполняемый граф
- **Память:** чекпоинтер (`InMemorySaver` , `SqliteSaver`) + `thread_id` — состояние между вызовами
- **Быстрый старт:** `create_react_agent` (модель + инструменты) или свой граф через `StateGraph`

Пример сценария: агент-интегратор

Цепочка: Google Sheets (задачи) → GitHub API (PR по задачам) → LLM (анализ) → Telegram (отчёт).

Агент объединяет данные из нескольких источников (Google Таблицы, GitHub), обрабатывает их через LLM и выдаёт структурированный результат (отчёт, уведомление).

Реализация сценария: 1 шаг (Google Sheets)

- API: `GET .../spreadsheets/{id}/values/{range}`
- Парсим: `ID` , описание, статус задач

Реализация сценария: 2 шаг (GitHub PR по ID задач)

- API: `GET /repos/:owner/:repo/pulls`
- Фильтр: заголовок или тело PR содержит ID задачи (например, `TASK-123`)



Реализация сценария: 3 шаг (анализ в LLM)

Промпт: список задач + список PR → попросить проверить:

- покрытие задач PR,
- несоответствия в описании,
- задачи без корректного PR.

Ответ получаем через API выбранной LLM.

Реализация сценария: 4 шаг (отчёт)

Пример ответа LLM: сводка по задачам (✅ покрыта PR, ⚠️ уточнить, 🚫 нет PR).

Куда отправить: консоль, email или Telegram-бот.

Идеи для расширения

-  Ветвления в LangGraph: если нет PR — узел «напоминание», иначе — отчёт
-  История в Notion / Airtable / Obsidian; чекпоинты + векторный поиск — долговременная память
-  Jira, ClickUp, Trello; **Human-in-the-loop** — пауза для одобрения перед действием
-  Визуализация графа: `graph.get_graph().draw_mermaid_png()` для отладки

-  Агент — модуль «принял → обработал → действовал»; LLM — «мозг» решений
-  **LangGraph**: StateGraph + узлы/рёбра, чекпоинты для памяти, `create_react_agent` или свой граф
-  Travily — низкоуровневые workflow; LangGraph — графовая модель поверх LangChain
-  Агенты экономят время, заменяют рутину и формируют основу AI-операторов

Вы получили новые инструменты — но опыт начинается с практики... 🔥

🎓 **Исследуйте** — пробуйте код, экспериментируйте, ошибайтесь

🔍 **Углубляйтесь** — каждая тема открывает новые возможности

🚀 **Создавайте** — даже маленький проект лучше теории

Это невозможно, пока вы это не сделаете!

Успехов!